Date: 24-10-2026

CO-2 - AT-2

SCENARIO BASED PROBLEM

COURES CODE: DSA0308

COUSRE NAME: NLP

Reg no: 192472313 **Name: Pavan Jana**

2: Morphological Parsing for Sentiment Analysis

Aim: To develop a Python program to identify prefixes, suffixes, and root words and analyze how word formation changes meaning in sentiment analysis.

Problem Statement: An AI-based sentiment analysis system is used to understand customer opinions in product reviews. Customers may use different forms of the same root word, such as *disagree, agreement,* and *agreeable*. These words share the root agree, but their meanings differ because of prefixes and suffixes.

The system needs to identify the root word, separate prefixes and suffixes, and classify the type of word formation. It must also explain how these changes affect meaning and convert the words into a common base form for further analysis. The Python program should display the original word, prefix, root, suffix, transformation type, meaning, and normalized form.

Algorithm:

1. Start the program.
2. Define the input words and morphological rules.
3. Identify the prefix, root, and suffix of each word.
4. Classify the morphological transformation.
5. Determine the meaning of each word.
6. Normalize each word to its root form.
7. Display the results.

8. Stop.

Program:

```
words = {
    "disagree": ("dis-", "agree", "", "Prefixation"),
    "agreement": ("", "agree", "-ment", "Suffixation"),
    "agreeable": ("", "agree", "-able", "Suffixation")
}

for word, (prefix, root, suffix, category) in words.items():
    print("\nWord:", word)
    print("Prefix:", prefix or "None")
    print("Root:", root)
    print("Suffix:", suffix or "None")
    print("Transformation:", category)
    print("Normalized form:", root)
```

Output:



Applications:

- Sentiment analysis
- Text classification
- Natural language processing
- Opinion mining and product review analysis

Result: The program identifies the root and affixes of the given words and normalizes them to agree.

Conclusion: The program successfully identifies the root word agree, detects the prefixes and suffixes, and classifies the transformations. All words are normalized to agree, helping process related words consistently while preserving their different meanings.

3: Morphology-Based Normalization

Aim: To develop a Python program that identifies roots and derivational suffixes and normalizes related words to a common base form for topic modeling and document clustering.

Problem Statement: A news aggregation system processes words such as *govern, government,* and *governance*. The system must identify their root forms, suffixes, and derivational levels and map them to a common representation to reduce vocabulary variations.

Algorithm:

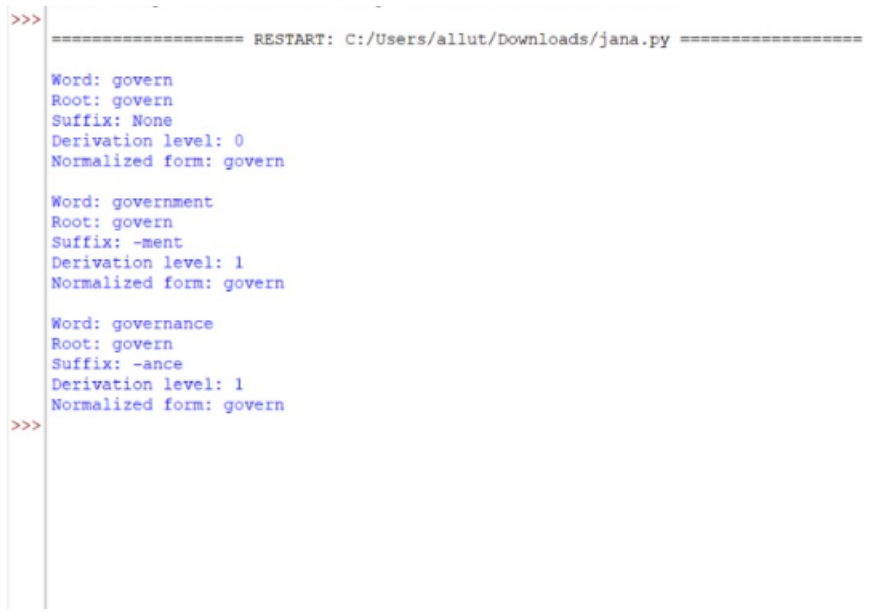
1. Start the program.
2. Define the input words and morphological rules.
3. Identify the root word and suffix.
4. Determine the derivational level.
5. Normalize each word to the common root.
6. Display the morphological breakdown.
7. Use the normalized words for topic modeling.
8. Stop.

Program:

```
words = {
    "govern": ("govern", "None", 0),
    "government": ("govern", "-ment", 1),
    "governance": ("govern", "-ance", 1)
}

for word, (root, suffix, level) in words.items():
    print("\nWord:", word)
    print("Root:", root)
    print("Suffix:", suffix)
    print("Derivation level:", level)
    print("Normalized form:", root)
```

Output:



Applications:

- News aggregation
- Topic modeling
- Document clustering
- Information retrieval
- Search engines and text mining

Result: The program identifies the root govern, detects the derivational suffixes, and normalizes all three words to govern.

Conclusion: The program successfully identifies the root govern, detects the suffixes -ment and -ance, and assigns derivational levels. All three words are normalized to govern, improving vocabulary consistency for topic modeling and document clustering.